

T3-Distancias-Caminos-Minimos.pdf



ivi4630



Matemática Discreta II



2º Grado en Ingeniería Informática



**Escuela Técnica Superior de Ingenieros Informáticos
Universidad Politécnica de Madrid**



[Accede al documento original](#)

TEMA 3. DISTANCIAS Y CAMINOS MÍNIMOS

lunes, 2 de diciembre de 2024 11:20

CONECTIVIDAD Y ORIENTABILIDAD

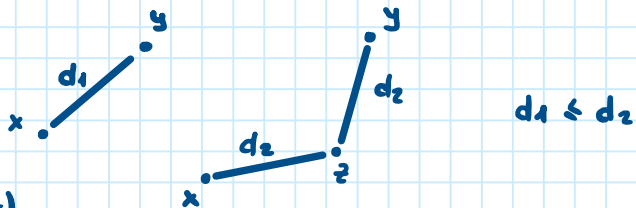
Grafo con pesos: Sea $G = (V, E, \rho, w)$ $w: E \mapsto \mathbb{R}$ un grafo con pesos. Sean u y v dos vértices de V y P un camino de u a v . La **LONGITUD** es P es la suma de los pesos de las aristas de P (es decir, el peso del subgrafo P)

$$d(x, y) = |x - y|, \quad x, y \in \mathbb{R}$$

$$(1) \quad d(x, y) \geq 0$$

$$(2) \quad d(x, y) = d(y, x)$$

$$(3) \quad d(x, y) \leq d(x, z) + d(y, z)$$

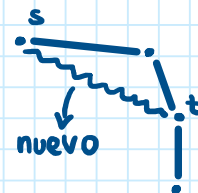


Algoritmo de Dijkstra

Obtiene los caminos de longitud (peso) mínima desde un vértice s hasta los restantes.

Es un algoritmo de tipo "voraz" que va incorporando nuevos vértices a un conjunto de vértices alcanzados mediante caminos mínimos.

En cada iteración incorporamos a dicho conjunto el vértice más próximo a él. Este proceso se repite hasta que todos los vértices están en el conjunto.



DESCRIPCIÓN DEL ALGORITMO

Entrada: Un grafo o digrafo ponderado $G = (V, E, \rho, w)$ y un vértice s de G .
El peso de la arista uv se indica por $w(u, v)$, poniendo $w(uv) = \infty$ si $uv \notin E$
→ El peso de cada arista del grafo debe ser no negativo.

Paso 1: Inicializamos el conjunto de vértices alcanzados mediante caminos mínimos ($T = \{s\}$) y la distancia t desde s hasta cada vértice de G se denota por $t(s) = d(s, s) = 0$, $t(z) = w(s, z) \forall z \neq s$

Paso 2: Elegir el vértice $u \notin T$ con t mínimo. Añadir u a T . Analizar cada arista uz con $z \notin T$ y actualizar la distancia $t(z) = \min\{t(z), t(u) + w(u, z)\}$

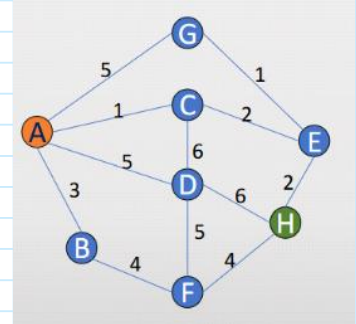
Paso 2: Elegir el vértice $u \notin T$ con t mínimo. Añadir u a T . Analizar cada arista uz con $z \notin T$ y actualizar la distancia $t(z) = \min\{t(z), t(u) + w(uz)\}$

Paso 3: Si $V-T \neq \emptyset$ y $\exists z \in V-T$ tal que $t(z) \neq \infty$, ir al paso 2.
EOC algoritmo terminado.

Salida: La distancia desde s a cada vértice V de G es $t(v)$
Si $V-T \neq \emptyset$, los vértices en $V-T$ no son accesibles desde s y tendrán $t(v) = +\infty$

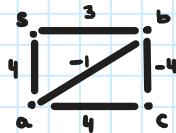
EJEMPLO TRANSPARENCIAS (Inicial = A, Final = H)

A	B	C	D	E	F	G	H	AA	VA
0	3,A	1,A	5,A	∞	∞	5,A	∞	AC	C
0	3,A	0	5,A	3,C	∞	5,A	∞	EC	E
0	3,A	0	5,A	0	∞	4,E	5,E	AB	B
0	0	0	5,A	0	7,B	4,E	5,E	EG	G
0	0	0	5,A	0	7,B	0	5,E	EH	H



La tabla representa la distancia desde A hasta cada vértice y el penúltimo vértice del camino.

PROBLEMA DEL ALGORITMO DE DIJKSTRA: CICLOS DE PESO NEGATIVO



s	a	b	c	AA	VA
0	-	-	-	-	s
0	4s	3s	∞	bs	b
0	2b	0	-1b	bc	c
0	2b	0	0	ba	a

$$d(s,b) = -\infty$$

$$d(s,a) = -\infty$$

Solución! $\Rightarrow$ Algoritmo de Bellman-Ford

ALGORITMO DE BELLMAN - FORD

Se aplica a grafos dirigidos. Es una variante del algoritmo de Dijkstra que admite pesos negativos en los arcos y permite detectar ciclos de peso negativo (que permitirían rebajar ilimitadamente la distancia entre vértices). No es un algoritmo de tipo varaz. El valor de $t(u)$ es una cota superior de la distancia $d(s,u)$ que se va actualizando hasta coincidir con la distancia. El algoritmo realiza $n-1$ actualizaciones para cada una de las aristas del digrafo.

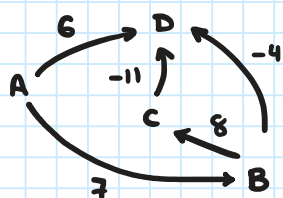
DESCRIPCIÓN DEL ALGORITMO

- Entrada: Un digrafo ponderado $G=(V,A)$ y un vértice $s \in V$. El peso del arco uv se indica por $w(u,v)$, (el peso puede ser negativo), y escribimos $w(u,v) = \infty$ si $uv \notin A$.
- Paso 1: Inicializar la cota superior de la distancia desde s hasta cada vértice z de G (esta cota superior se denomina $t(z)$): $t(s) = d(s,s) = 0$
 $t(z) = \infty \quad \forall z \neq s$
- Paso 2: Repetir n veces ($n = n^o$ vértices) lo siguiente:
Para cada arco $e = uz \in A$ actualizar $t(z) \quad t(z) = \min\{t(z), t(u) + w(uz)\}$
- Salida: Si al efectuar la n -ésima iteración no hay arcos que produzcan cambios en t , entonces $t(u) = d(s,u)$ para cada $u \in V$. EOC, el digrafo contiene un ciclo de peso negativo.

EJEMPLO TRANSPARENCIAS (1)



WUOLAH



4 iteraciones (NI-1)

1ª iteración

ARISTA/VERT	A	B	C	D
AB	0	7	∞	∞
BD	0	7	∞	3
BC	0	7	15	3
CD	0	7	15	3
AD	0	7	15	3

$$t(B) = \min\{0, t(A) + w(AB)\}$$

$$t(D) = \min\{0, t(B) + w(BD)\}$$

actualizo aquí

2ª iteración

ARISTA/VERT	A	B	C	D
AB	0	7	15	3
BD	0	7	15	3
BC	0	7	15	3
CD	0	7	15	3
AD	0	7	15	3

La tercera y cuarta iteración van a ser iguales, por lo que no las escribo.

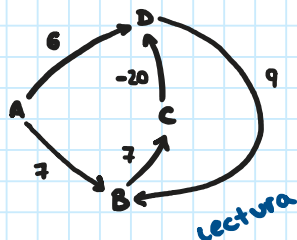
No produce modificación en la tabla de la 2ª iteración $\Rightarrow$ no hay ciclos de peso negativo y obtenemos los mismos resultados que obtenemos al aplicar Dijkstra

$$d(A,B)=7 \quad d(A,C)=15 \quad d(A,D)=3$$

X la fórmula sería algo como:

$$t(B) = \min\{0, t(A) + w(AB)\}$$

EJEMPLO TRANSPARENCIAS (2)



lectura

1ª iteración

ARIST/VERT	A	B	C	D
AD	0	∞	∞	6
DB	0	15	∞	6
BC	0	15	22	6
CD	0	15	22	2
AB	0	7	22	2

2ª iteración

ARIST/VERT	A	B	C	D
AD	0	7	22	2
DB	0	7	22	2
* BC	0	7	14	2
CD	0	7	14	-6
AB	0	7	14	-6

* {BC act C, llegar a B cuesta 7 y BC pesa 7, $7+7=14$ }

3ª iteración

AR/VER	A	B	C	D
AD	0	7	14	-6
DB	0	3	14	-6
BC	0	3	10	-6
CD	0	3	10	-10
AB	0	3	10	-10

AR/VER	A	B	C	D
AD	0	3	10	-10
DB	0	-1	10	-10
BC				
CD				
AB				

↓ NO SIGO ↓

Ya hay cambio, no quedan más iteraciones, por lo que acabo aquí.

Cambia!! por lo que hay un ciclo de pesos negativos.

DIKJSTRA VS BELLMAN - FORD

→ En principio NO DIRIGIDOS

DIKJSTRA: Es para grafos sin pesos negativos

Generamos T incorporando a cada paso el vértice más próximo al origen (primer vértice). Al añadir un vértice u recalculamos la distancia para cada vértice $z \notin T$ tal que $uz \in A$ como $t(z) = \min\{t(z), t(u) + w(uz)\}$ es decir, o actualizamos o dejamos tal cual.

BELLMAN-FORD: Es para grafos dirigidos con pesos arbitrarios.

Examinamos los arcos de uno a uno y repetimos todo el proceso n veces ($n = |V|$). Al considerar el arco $uz \in A$ actualizamos la distancia al vértice

(FOTO)

EXCENTRICIDAD, CENTRO Y DIÁMETRO DE UN GRAFO

Def:

(1) Dado un grafo o digrafo $G=(V,E,b)$, dos vértices u y v de G y un camino P en G entre u y v, se denomina longitud del camino P al número de aristas (o arcos) de P.

WUOLAH

(2) Se denomina distancia de u a v , denotada por $d(u,v)$ a la mínima longitud de los caminos que unen u con v . Si no existe camino entre u y v , decimos que la distancia $d(u,v) = +\infty$

(3) Si G es ponderado $G=(V,E,p,w)$ en el que cada arista $e \in A$ tiene un peso $w(e)$, el concepto de la longitud o peso de un camino P se aplica a la suma de los pesos de las aristas de P i.e.

$$\text{long}(P) = \sum_{e \in P} w(e)$$

Def: Sea $G=(V,A,p)$ un grafo conexo

(4) La excentricidad de un vértice $v \in V$ es $e(v) = \max\{d(v,z); z \in V\}$ → Máxima distancia desde un vértice a otro cualquiera.

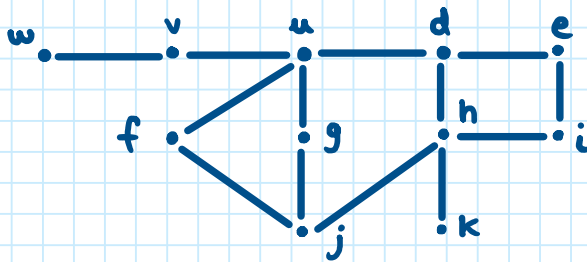
(5) El radio de un grafo es $\text{rad}(G) = \min\{e(v) : v \in V\}$ → IDEM: Máx → Min sustituye.

(6) El diámetro del grafo es $\text{diam}(G) = \max\{e(v) : v \in V\}$

(7) La distancia total de un vértice $v \in V$ es $dt(v) = \sum_{z \in V} d(v,z)$ → Suma de las excentricidades.

Teorema: Dado un grafo conexo G se cumple que $\text{rad}(G) \leq \text{diam}(G) \leq 2\text{rad}(G)$

EJEMPLO TRANSPARENCIAS



$d(u,v)$	w	v	u	d	e	f	g	h	i	j	k	$e(v)$	$dt(v)$
w	0	1	2	3	4	3	3	4	5	4	5	5	34
v	1	0	1	2	3	2	2	3	4	3	4	4	25
u	2	1	0										
d	3	2		0									
e		3			0								
f						0							
g							0						
h								0					
i									0				
j										0			
k											0		

subgrafos

(8) El centro de un grafo conexo G es el subgrafo inducido por el conjunto de vértices de excentricidad

(8) El centro de un grafo conexo G es el subgrafo inducido por el conjunto de vértices de excentricidad mínima.

(9) La periferia de un grafo conexo G es el subgrafo inducido por el conjunto de vértices de excentricidad máxima.

(10) La mediana de un grafo conexo G es el subgrafo inducido por el conjunto de vértices de distancia total máxima.

TEOREMA DE JORDAN

El centro de cualquier árbol consta de uno o dos vértices (ie, es un vértice o arista)

A continuación se muestra un algoritmo para el cálculo del centro de un árbol.

P1. Haces $H=T$ donde T es un árbol

P2. Si $H = K_1$ o $K_2 \Rightarrow$ El centro es $H = \{K_1 \cdot, K_2 \text{ --- } \cdot\}$

P3. En caso contrario, borramos todos los vértices de grado 1 de H para obtener un árbol H' . Hacemos $H = H'$ y volvemos a P2.

CONECTIVIDAD

Def: La conectividad de un grafo G , denotada por $K(G)$ es el mínimo número de vértices que se deben eliminar de G para producir un grafo no conexo o un grafo trivial. Se dice que un grafo es k -conexo si $K(G) \geq k$

OBS

- Un grafo con vértices corte tiene conectividad 1
- Un ciclo con vértices corte tiene conectividad 2
- El grafo completo K_n tiene conectividad de aristas $n-1$
- En general, en un grafo G se tiene $\lambda(G) \leq \sigma(G)$

11.



$$\delta(G) = 3$$

$$\lambda(G) = 3$$

$$K(G) = 3$$



$$\delta(G) = 4$$

$$\lambda(G) = 4$$

$$K(G) = 3$$

Revisa los fotos del día 7-11, creo que me he dejado un algoritmo

